HW3 实验报告（PyTorch 版本）

1. 实验方案与代码补全说明

本次实验选择 **PyTorch** 方法完成古诗生成任务，代码目录为 `tangshi_for_pytorch`。

根据题目要求，我在 `tangshi_for_pytorch/rnn.py` 中补全了两处关键代码：

1. 在 `RNN_model.__init__` 中定义两层 LSTM 网络 `self.rnn_lstm`：
   - 输入维度：`embedding_dim`
   - 隐藏层维度：`lstm_hidden_dim`
   - 层数：`num_layers=2`
   - 张量格式：`batch_first=True`（即输入输出使用 `(batch, seq, feature)`）
2. 在 `RNN_model.forward` 中完成 LSTM 前向传播：
   - 初始化全零隐状态 `h_0` 和细胞状态 `c_0`
   - 状态维度为 `(2, batch_size, lstm_hidden_dim)`，其中 2 对应两层 LSTM
   - 将 `batch_input` 与初始状态送入 LSTM，得到隐藏输出 `output`

上述补全使模型能够从词向量序列中提取时序特征，并继续通过全连接层与 `LogSoftmax` 完成下一词预测。

---

2. RNN、LSTM、GRU 模型原理说明

2.1 RNN（Recurrent Neural Network）

RNN 通过循环结构处理序列，当前时刻不仅依赖输入 x_t，还依赖上一时刻隐藏状态 h_{t-1}：

$$h_t = \phi(W_{xh}x_t + W_{hh}h_{t-1} + b_h)$$

$$y_t = W_{hy}h_t + b_y$$

其中 ϕ 常为 `tanh` 或 `ReLU`。RNN 能表达时间依赖，但在长序列上容易出现梯度消失/爆炸，难以学习长期依赖。

2.2 LSTM（Long Short-Term Memory）

LSTM 在 RNN 基础上增加“门控机制”和“细胞状态” c_t，缓解长期依赖问题。其核心计算如下：

$$f_t = \sigma(W_f[h_{t-1}, x_t] + b_f) \quad (\text{遗忘门})$$

$$i_t = \sigma(W_i[h_{t-1}, x_t] + b_i) \quad (\text{输入门})$$

$$\tilde{c}_t = \tanh(W_c[h_{t-1}, x_t] + b_c) \quad (\text{候选记忆})$$

$$c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t$$

$$o_t = \sigma(W_o[h_{t-1}, x_t] + b_o) \quad (\text{输出门})$$

$$h_t = o_t \odot \tanh(c_t)$$

LSTM 通过门控选择“保留什么、写入什么、输出什么”，因此比基础 RNN 更适合文本生成任务。

2.3 GRU（Gated Recurrent Unit)

GRU 可视为 LSTM 的简化版本，将细胞状态与隐藏状态合并，参数更少、训练更快。核心公式：

$$z_t = \sigma(W_z[h_{t-1}, x_t] + b_z) \quad (\text{更新门})$$

$$r_t = \sigma(W_r[h_{t-1}, x_t] + b_r) \quad (\text{重置门})$$

$$\tilde{h}_t = \tanh(W_h[r_t \odot h_{t-1}, x_t] + b_h)$$

$$h_t = (1 - z_t) \odot h_{t-1} + z_t \odot \tilde{h}_t$$

与 LSTM 相比，GRU 结构更紧凑，常在数据量较小或需要更快训练时表现良好。

---

3. 诗歌生成过程叙述

本实验的古诗生成流程可以分为“数据预处理 -> 模型训练 -> 条件生成”三步。

3.1 数据预处理与编码

1. 读取诗歌语料文件（`poems.txt`），按“标题:正文”解析每一行文本。
2. 对正文进行清洗，去除空格，并过滤异常字符或长度不合适的样本（过短或过长）。
3. 在每首诗前后分别加入起始标记 `G` 和结束标记 `E`，用于提示模型“从哪里开始生成、何时停止”。
4. 统计语料中所有字符的出现频率，构建词表 `vocabularies` 以及字符到索引的映射 `word_to_int`。
5. 将每首诗从字符序列转换为索引序列，得到可用于神经网络训练的数字化数据。

经过以上步骤，原始文本被转化为模型可处理的离散序列输入。

3.2 序列建模与训练

1. 网络结构由“词向量层 + 两层 LSTM + 全连接层 + LogSoftmax”组成。
2. 训练时采用自回归目标：输入序列为 $x_1, x_2, \ldots, x_T$，监督信号为右移一位后的序列

$$y_t = x_{t+1}$$

   即让模型学习“给定前文预测下一个字”。
3. 损失函数使用 `NLLLoss`，优化器使用 `RMSprop`，并通过梯度裁剪限制梯度范数，提升训练稳定性。
4. 在每个 epoch 的若干 batch 后保存 checkpoint（包含模型参数与词表映射），保证后续推理时训练/推理词表一致。
5. 训练阶段在设备可用时启用 GPU（CUDA）加速，缩短训练时间。

通过不断迭代，模型逐步学习到古诗中常见的字词搭配、句式结构与韵律模式。

3.3 给定首字的诗歌生成

1. 推理时先加载训练好的 checkpoint，并恢复与训练一致的词表映射。
2. 以给定首字（如“日、红、山、夜、湖、海、月”）作为初始输入。
3. 模型输出下一个字的概率分布，取概率最大的字作为当前时刻预测结果。
4. 将新预测字拼接到已有序列，再次送入模型，循环执行“预测 -> 拼接 -> 再预测”。
5. 当生成到结束标记 `E` 或达到最大长度上限时停止，得到完整诗句。

该过程本质上是条件自回归生成：每一步都依赖已生成的历史上下文，从而逐字扩展为整首诗。

4. 训练与生成结果展示

4.1 训练过程截图

训练阶段按作业要求提供两张截图，展示模型在训练过程中的预测输出与损失变化情况。

```
(py312_GNN) E:\学习\大三下\深度学习\HW\HW3\tangshi_for_pytorch>python main.py
finish  loadding data
using device: cuda
prediction [4084, 4084, 4084, 4084, 4084, 4084, 2574]
p_y        [28, 546, 104, 718, 1, 3, 3]
********************************
epoch   0 batch number 0 loss is:  8.717823028564453
finish  save model
prediction [5, 3, 3, 3, 3, 3, 3, 3, 3, 3, 3, 3]
p_y        [267, 2888, 267, 2045, 0, 26, 1067, 26, 349, 1, 3, 3]
********************************
epoch   0 batch number 1 loss is:  7.482438087463379
prediction [3, 3, 3, 3, 3, 3, 3, 3, 3, 3, 3, 3, 3, 3]
p_y        [617, 564, 80, 601, 132, 0, 9, 184, 2007, 373, 305, 1, 3, 3]
********************************
epoch   0 batch number 2 loss is:  8.139092445373535
prediction [0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0]
p_y        [125, 9, 3862, 695, 16, 1721, 1721, 0, 373, 591, 246, 695, 900, 80, 431, 1, 3, 3]
********************************
epoch   0 batch number 3 loss is:  7.362464904785156
prediction [3, 3, 3, 3, 3, 3, 3, 3, 3, 3, 3, 3, 3, 3, 3, 3, 3, 3, 3, 3]
p_y        [1539, 1058, 3996, 64, 0, 793, 977, 550, 381, 1, 3867, 3285, 972, 1287, 0, 1541, 1541, 1779, 1779, 1, 3, 3]
********************************
epoch   0 batch number 4 loss is:  7.334989070892334
prediction [1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1]
p_y        [194, 348, 66, 89, 73, 0, 574, 21, 855, 101, 1013, 1, 293, 162, 1283, 117, 383, 0, 25, 74, 4, 253, 611, 1, 3, 3]
********************************
epoch   0 batch number 5 loss is:  7.246746063232422
prediction [3, 3, 0, 0, 0, 3, 0, 2824, 826, 826, 5616, 826, 880, 2193, 979, 979, 979, 2150, 5945, 1992, 186, 186, 186, 2143, 5269, 116]
p_y        [349, 351, 456, 220, 850, 0, 304, 146, 195, 350, 576, 1, 264, 178, 35, 323, 189, 0, 243, 12, 273, 61, 116, 1, 3, 3]
********************************
```

```
*****************************
epoch   29 batch number 341 loss is:  6.199408531188965
prediction [10, 72, 155, 4, 300, 0, 10, 108, 4, 174, 89, 1, 10, 45, 9, 9, 51
04, 36, 410, 5, 1, 498, 5, 9, 69, 43, 0, 40, 184, 4, 182, 58, 1, 3, 3]
b_y        [562, 118, 30, 99, 29, 0, 473, 2716, 1944, 298, 2495, 1, 536, 140
4, 957, 202, 743, 1, 13, 35, 115, 193, 274, 0, 22, 520, 1103, 321, 236, 1, 1
*****************************
epoch   29 batch number 342 loss is:  6.242142200469971
prediction [10, 284, 9, 15, 117, 0, 84, 239, 10, 10, 150, 1, 10, 181, 240, 4
74, 57, 0, 6, 52, 82, 126, 162, 1, 15, 632, 4, 26, 20, 0, 4, 233, 4, 147, 10
b_y        [493, 169, 505, 102, 909, 0, 934, 1577, 10, 701, 279, 1, 989, 824
 409, 370, 1, 1150, 1641, 716, 1543, 816, 0, 217, 979, 1020, 126, 2431, 1, 2
*****************************
epoch   29 batch number 343 loss is:  6.274418830871582
prediction [10, 70, 27, 82, 508, 0, 10, 56, 4, 265, 417, 1, 10, 77, 118, 343
, 344, 0, 211, 16, 4, 30, 79, 1, 3, 41, 9, 115, 43, 0, 40, 27, 36, 44, 5, 1,
b_y        [10, 410, 708, 926, 1773, 0, 141, 3026, 1493, 669, 849, 1, 811, 1
 35, 461, 166, 7, 1, 564, 580, 30, 560, 2038, 0, 1464, 1817, 1981, 685, 40,
*****************************
epoch   29 batch number 344 loss is:  6.2069573402404785
prediction [10, 572, 4, 356, 108, 0, 0, 35, 4, 31, 395, 1, 7, 35, 4, 19, 51,
 7, 77, 82, 7, 1, 3, 56, 4, 17, 44, 0, 4, 126, 4, 7, 40, 1, 3, 3]
b_y        [228, 2381, 21, 1185, 763, 0, 305, 526, 159, 1500, 1358, 1, 668,
40, 468, 488, 228, 698, 1, 46, 43, 47, 33, 16, 0, 1345, 205, 49, 197, 7, 1,
*****************************
epoch   29 batch number 345 loss is:  6.387474060058594
prediction [10, 115, 4, 16, 383, 0, 10, 72, 82, 178, 79, 1, 10, 95, 10, 104,
4, 19, 10, 17, 4, 11, 40, 1, 3, 35, 4, 44, 69, 26, 104, 0, 4, 19, 84, 35, 4,
b_y        [1472, 115, 959, 63, 28, 0, 74, 202, 166, 178, 167, 1, 103, 62, 3
 113, 281, 367, 478, 255, 134, 1104, 0, 21, 229, 50, 17, 36, 47, 199, 1, 143
*****************************
epoch   29 batch number 346 loss is:  6.516101360321045
prediction [10, 5, 4, 15, 0, 0, 8, 0, 10, 30, 22, 11, 4, 1, 45, 1, 10, 56, 1
 51, 0, 4, 140, 84, 5, 4, 48, 94, 1, 4, 56, 4, 30, 5, 23, 44, 0, 10, 117, 5,
b_y        [341, 5, 19, 96, 450, 68, 269, 0, 1269, 893, 329, 384, 932, 287,
 145, 249, 1365, 1, 10, 59, 110, 264, 337, 474, 248, 0, 275, 1099, 9, 225, 2
*****************************
epoch   29 batch number 347 loss is:  6.373894691467285
```

4.2 诗歌生成结果截图

以“日、红、山、夜、湖、海、月”等字作为起始词进行生成，结果截图如下：

```
(py312_GNN) E:\学习\大三下\深度学习\HW\HW3\tangshi_for_pytorch>python main.py
日边酒，一树风光入玉杯。
自有君王不至，如能有此中人。
愿得君王在，何人更此生。
风光不足绝，风景入红霞。
舞罢人相问，相逢不得自如。
何事一杯酒，何人不相传。
不知何日在天子，不觉人家在玉台。
玉人不得在，此日无人知。
愿与东林下，何人不自知。
不知何日不知月，不觉人家不自伤。
何如一杯酒，不得不知行。
月微微，一树风光入玉杯。
自有天台何足问，不知何日更何人。
愿得君王不至，无人知玉石台。
```

---

5. 实验总结

本实验基于 PyTorch 完成了古诗生成任务，核心工作包括：

1. 按要求补全 `rnn.py` 中两处关键代码，完成两层 LSTM 的构建与前向传播。
2. 梳理并说明了 RNN、LSTM、GRU 的结构原理与公式差异。
3. 完整实现了从语料预处理、序列建模训练到给定起始字生成诗句的全过程。
4. 提供了训练截图与“日、红、山、夜、湖、海、月”起始词的生成结果截图，满足作业展示要求。

从实验结果看，LSTM 能够学习到一定的古诗语言模式，在字词衔接和句式结构上具备可读性；但受训练轮次、数据质量与解码策略影响，部分输出仍存在语义跳跃与意境不连贯的问题。后续可通过以下方向进一步改进：

1. 增加训练轮次并调节学习率、隐藏层维度等超参数。
2. 引入更稳定的采样策略（如温度采样、top-k/top-p），提升生成多样性与自然度。
3. 融合押韵、平仄或主题约束，增强诗歌的文学性与可控性。